

8. Choosing the Correct Event Schema Design in Event-Driven Microservices

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Model events using event storming
- Using and understanding which information benefits from being transferred in event headers
- Designing small events and their impacts
- Designing large and denormalized events and their impacts
- Understanding the importance of involving consumers in the event schema design
- Avoiding breaking changes and how to evolve event schema

Darwin once said it isn't the stronger or most intelligent species that survive but the most adaptable. Events are much like it; there isn't a one-fits-all rule to design events; they often need to be adapted to each use case and have the requirements of the consuming application in mind. Events are the essence of every event-driven architecture; as we discussed in the previous chapters, they are a meaningful and sustainable way to share data throughout the company. They represent the business's very history retaining its value and exposing it in a decoupled and scalable way. Exposing data in a streaming and scalable medium provides the foundation for powerful use cases. Events are the blood flow in an event-driven architecture; designing the correct event schema for each use case becomes a key design consideration.

We discussed the advantages of having durable event brokers and retaining the history of events. It's a powerful concept, until you have to insert a breaking change in the event schema, and then history seems a lot

less valuable compared to the challenge of handling multiple event schemas. This chapter will also discuss how to approach schema changes and work through a use case illustrating how we can approach schema evolution.

Designing events is often a gray area; there are standard best practices, but following them blindly can severely impact the architecture and produce complicated workarounds. Understanding the implications of the different design approaches, weighing the tradeoffs, and making deliberate decisions is fundamental to achieve a maintainable and scalable solution. This chapter will discuss these tradeoffs and work through a use case using different approaches explaining the advantages and downsides. Although it won't give you a silver bullet to always design the correct schema, it will provide you with the considerations to make sustained decisions and choose the appropriate implications for each use case.

8.1 Event Storming and Event-Driven Microservices

Event storming is a technique to model complex business processes that has high synergy with event-driven architectures. It shares some of the DDD concepts, but it's focused on a timeline of events that we can often directly translate to code. The full implementation of event storming is out of this book's scope, and it's fully addressed in Alberto Brandolini's book,¹ its original creator. This section will give you the foundations for exploring it further and why event storming can be relevant in modeling an event-driven architecture and its events.

Event storming is a tool to gain a higher level of understanding of a complex domain and promote shared understanding around the domain's business processes. It involves modeling a business process as a flow of events represented by stickies on a whiteboard. The technical and business experts use different colored stickers to model the sequence of domain events, commands, components, external systems, and questions in a business process. In a relatively short session, around one hour, a rather complex business process can be modeled, as long as the right peo-

ple are in the room and the appropriate mediation is provided. Further sessions can be scheduled to dive down on unclear subjects or to further detail higher-level flows.

It's also about gathering the right people in the same room and doing temporal modeling of the business process. Instead of focusing on a structure like typical modeling approaches (like UML, sequence diagrams, etc.), focuses on mapping the timeline of successive events that happen in the process. It can be highly beneficial as a tool for collective learning and modeling the whole end-to-end process. It also helps identify possible early gaps, blockers, misunderstandings, and gray areas that might become a problem later in the project.

So, how can we apply it? The crucial point about event storming is the presence of the right people in the event storming session. There should be elements of the development team and the domain experts. If we don't have the correct people in the session, we should postpone it and guarantee everyone is present. Once we gather all relevant people and clearly define the session's objective (e.g., event map the order fulfillment process), we should ask the participants to model the process with events. Events are represented by orange stickies and should be placed on a large open area. A large area is important; we want the participants to map the process and not discuss it fruitlessly without physical support. A cramped space to put the stickies limits the thought process; discussions without physical support often have no real conclusions or action points; we want to avoid that. Then the participants should map the process as a sequence of events. Figure [8-1](#) illustrates the sequence of events in an order fulfillment flow.

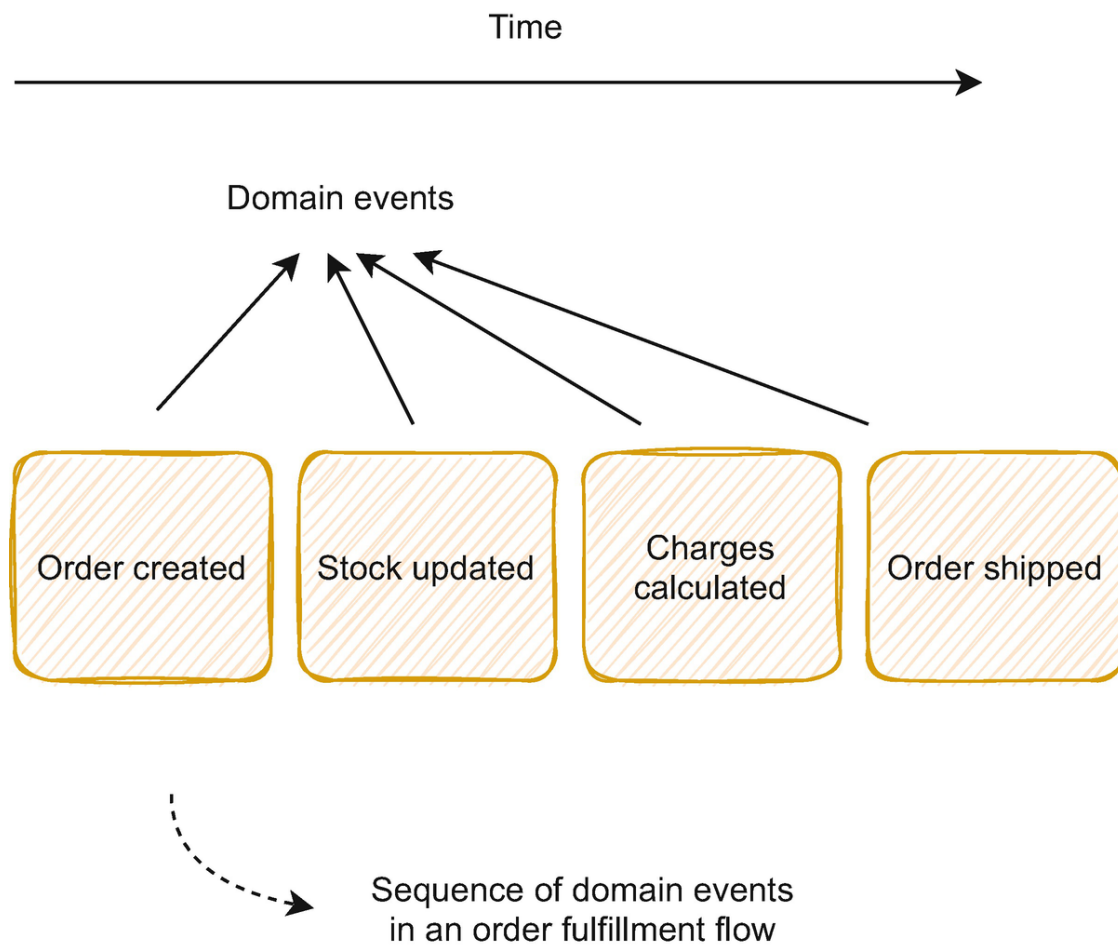


Figure 8-1 Domain events example of an order fulfillment flow

The most valuable first step is to model the business process as a sequence of domain events. The sequence by itself vastly increases the knowledge of the process of everyone in the session. An arguably good first interaction is to have the moderator ask the participants an event of the business process and write it down, which typically helps break the ice and kick off the activity. Domain events are a business occurrence that is relevant for the business experts. Domain experts can explain the process according to these events as they flow in a timeline. Presenting the business process with this mindset provides a valuable framework for storytelling and is easy for anyone, even without a technical background, to understand. An important moderation tip is to let the domain experts map the events, and the developers explore their meaning instead of developers guiding the thought process.

Once you have the main flow of domain events, you can add the other typical elements of an event storming session:

- Domain events, which we just discussed (typically represented by orange stickies)
- User commands, a user action (typically represented by blue stickies)
- Questions, risks, assumptions, and concerns, which we use to signal possible gaps or parts of the process that need further analysis (typically represented by pink stickies)
- Persona/actor, who initiated the action (typically represented by yellow stickies)
- External systems, events from an external system (typically represented by light blue/cyan stickies)
- Data, data consultation to make a decision or a read model (typically represented by light green stickies)

I find that typically it's best to map the domain event sequence and then iterate on adding detail about the other types of artifacts. Eventually, it can evolve to something similar to Figure [8-2](#).

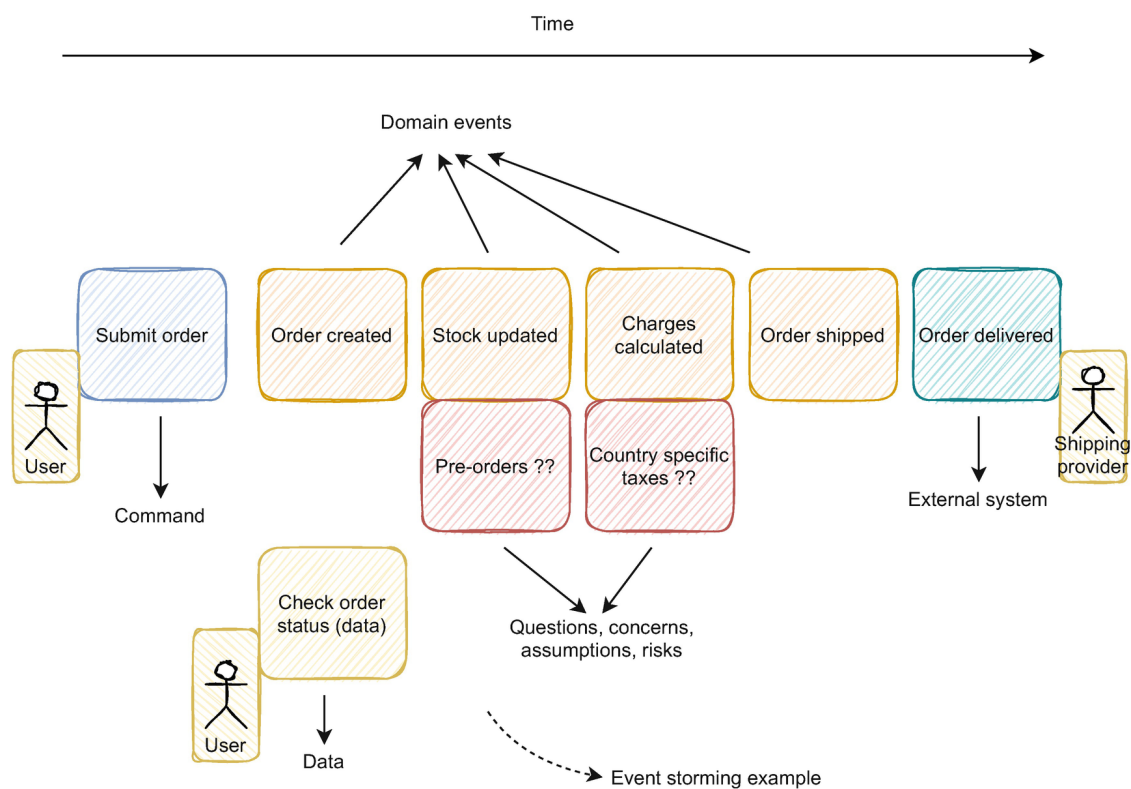


Figure 8-2 Event storming example of an order fulfillment flow with additional artifacts

An important detail is the question/pink stickies; if the participants get stuck discussing a subject, write it on one of those stickies. It promotes everyone's collaboration, and if the discussion takes too long, we can mark it and move on. Also, following the typical diverge, explore, and

converge approach can be beneficial. You want the participants to discuss the overall process and the sequence of events (diverge and explore). Next, try to make sense of the confusion and group everyone to organize the event flow (converge).

This process promotes active collaboration to discover the right thing to do. In large microservice architectures, developers tend to implement projects focused on a single or few services, and it is easy to lose the whole picture of the end-to-end process. It is also easy to do what they are told, which might not be the right thing. Event storming greatly benefits the dissemination of knowledge and clarifies the main objective of the project.

One thing DDD often mentions is the domain expert. In fact, in large organizations, there isn't one domain expert; typically, there are knowledge silos. In each silo, there is one or a few domain experts specialized in that silo; there is hardly anyone with knowledge of all silos. Event storming shines in these environments where we can bring each domain expert to the session. Each person in the session individually might not know everything, but everyone's combined knowledge provides to be exceptionally insightful. It also provides a platform for group validation, much like a code review to a process modeling done by domain experts.

Event storming has high synergy with event-driven architectures since the domain events often map to the technical domain events, and what better way to design those events than with the domain experts' input? It also highlights some beneficial architectural decisions. For example, in Figure [8-2](#), we have an order created domain event and a check order status data model; this might directly translate to a CQRS pattern. It often proves to be a smooth translation from the model to the actual implementation.

For further details and moderation guidelines, we highly recommend Alberto Brandolini's book; he also has a great talk at a NewCrafts conference worth checking out.^{[2](#)} Paul Rayner also has a fascinating talk with several moderation tips and detailing event storming concepts.^{[3](#)}

8.1.1 What Are the Limitations of Event Storming?

Event storming might not be suitable for simple and straightforward domains. For example, if a domain can be clearly expressed by a CRUD implementation, modeling it by events might not be adequate. However, much like DDD, the understanding and knowledge acquired by the exercise are often extremely valuable.

In more extensive processes and domains with intricate interactions, moderation becomes especially relevant. Experienced moderation can be hard to find and might be a challenge for people trying out event storming⁴ for the first time. The book *Game Storming* can help with some frameworks.

One challenge is often having the right people in the session; the domain experts are often hard to pin down. The presence of the right domain experts is usually a do or break requirement; we need the right people in the session to be productive. It also becomes an issue with distributed teams; having people physically in the same room produces a unique atmosphere and promotes a different type of collaboration than having people collaborating remotely. You might still benefit from the exercise, but it might not be as valuable.

8.2 Event Schema: Headers and Envelopes

The event's payload is obviously the pivotal part of the event. However, an often overlooked detail is the event's headers. In some use cases, the event benefits from having some generic properties available in the headers or the event's envelope. This section will discuss the advantages and drawbacks of having an envelope and will work through a use case illustrating which information can be relevant to add to an envelope or headers.

There is some information relevant for every event, independent of their content or origin. This type of generic information or metadata can be shared using a generic structure like an event's headers or an envelope, as illustrated in Figure 8-3. Having standardized properties across the organization can help generic processes or utilities (like tracing) without digging down to the event's payload. It also provides better segregation; the event's payload will only have the information related to the

occurrence and the domain, while we can segregate infrastructure information with a more abstract nature to an isolated structure.

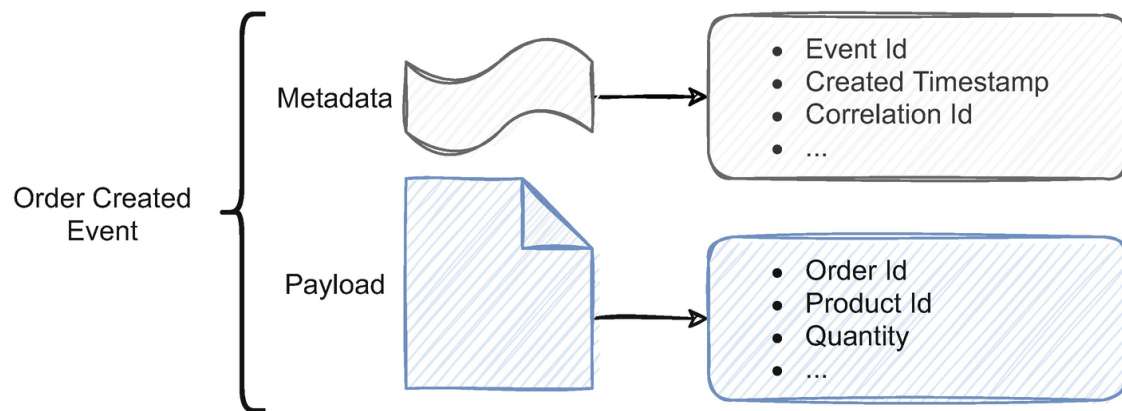


Figure 8-3 Segregation of the event's metadata and payload

The metadata provides contextual information about the event, while the payload provides the event's information. It's similar to an HTTP request; the body of the request carries the request's payload, and the HTTP headers have more contextual information about the request. The event's metadata can be transferred using an envelope or through message headers. A message envelope is like a wrapper around or inside the event's payload with a standard structure common to all events. Most message brokers also support message headers, and much like HTTP headers, they are often modeled in a key-value approach with the flexibility to define custom values.

8.2.1 Headers vs. Envelopes in Event Schema

There are typically two approaches to implement event envelopes, either by a composed or flat structure. A flat structure adds the information next to the event's payload, with a property containing all metadata information. The composed structure is more popular and resembles an actual envelope; the envelope is a wrapper with contextual information and an abstract payload. Listing [8-1](#) illustrates the two types of envelopes applied to an order created event.

```
1   Composed OrderCreatedEvent
2   {
3       Metadata:
4       {
5           EventId: 1231,
6           Timestamp: "2021-01-30T11:41:21.442Z"
7       },
8       Payload:
```



```

9      {
10          OrderId: 3621,
11          ProductId: 2184,
12          Quantity: 2,
13          UserId: 164
14      }
15  }
16
17  Flat OrderCreatedEvent
18  {
19      Metadata: [{"EventId": 1231}, { "Timestamp": "2021-
01-30T11:41:21.442Z"}],
20      OrderId: 3621,
21      ProductId: 2184,
22      Quantity: 2,
23      UserId: 164
24  }

```

Listing 8-1 Composed vs. flat envelope structure

Every event with a flat structure has a property specific to the event's metadata. While the composed structure segregates it at a higher level, the event has two root fields: one for the metadata and one for the payload. The metadata in the composed field can also be a generic array of dynamic properties, although there are advantages of defining static metadata fields. For example, if we define a standard envelope for the company, we can enforce specific fields like the correlation id or the user id for audit purposes. It also incites collaboration on the envelope definition (although this can be an advantage or drawback depending on the company).

However, the envelope tends to get in the way of the event and sometimes can be challenging to deal with. It introduces extra information to serialize and deserialize having a performance hit; also changes in the envelope tend to be difficult to manage. If changing an event's schema requires a lot of communication and alignment, imagine changing the envelope's schema when all events share the same envelope. Also it can become hard to simultaneously handle sources that use envelopes and sources that don't use envelopes, often leading to additional code overhead. There is an interesting Confluent article⁵ further exploring the different envelopes worth checking out.

Headers often tend to ease this process; we can share common contextual information in event headers and typically offer better segregation of concepts; the payload has the event and the headers have the metadata. Headers are a common feature in most brokers, and since they are dynamic and optional, they make different use cases easier to handle. To guarantee coherence across services, most use cases might benefit from a common definition with the custom headers and the ones the company wants to implement. However, it is harder to enforce the presence of specific headers; if a requirement requires the absolute presence of given information across all services, then an envelope might help to enforce that requirement. It depends on the use case, but overall, using headers tends to be a more flexible and less intrusive approach.

8.2.2 Relevant Contextual Information in Events

Envelopes and headers are great for sharing common contextual information. But what kind of information is relevant to use as metadata or contextual data? It heavily depends on the use case, and you might even find it helpful to define specific custom headers only relevant for your domain or business. However, there's often common information useful for most use cases; this subsection will detail some common use cases of contextual information. This way, you can reason whether it makes sense for your use case and which kind of information is typically sent in the metadata.

HTTP has a reasonably detailed definition in terms of requests, responses, and even common headers. This definition provides interoperability across systems, companies, and services. Having a clear contract to adhere to and some assumptions on how the system works facilitates communication and developments. Events, being the medium for communication across companies, especially in event-driven architectures, could arguably benefit from a similar definition. CloudEvents⁶ is an approach giving this definition and provides a specification for describing event data. It provides some examples of mandatory and optional contextual attributes. Here are some examples typically relevant to most use cases:

- **Id:** The event's id. Having an event id is often useful for debugging, tracing, and even managing idempotency (like we mentioned in Chapter 6). It has to be unique per event; generating a unique incremental

id can be troublesome; using a UUID (universally unique identifier) can be helpful.

- Correlation id: An id associating different operations together. For example, when we discussed the choreography pattern, several services choreographed to fulfill an order. The order service created the order, the inventory service managed the order's stock, the pricing service calculated the fees, etc. Each service would publish an event; having a correlation id across all services is a way to quickly obtain all operations in the scope of a single order.
- Source: The service and operation that published the event. Especially relevant when different processes in a single service can generate an event.
- Version: The aggregate's version. Often valuable for managing idempotency and for debugging purposes.
- Timestamp: The date when the event occurred. Also relevant for debugging and tracing purposes. It can be helpful to manage concurrency when a version is absent.
- Priority: The importance of the event. We can assign events to different streams with different priorities (as we discussed in Chapter [6](#)).
- User id: Id of the user that triggered the event. Often useful for auditing and debugging purposes.

8.3 Town Crier Events Pattern

Several questions can arise when designing event schema. A common design question is how much information the event should have and how large it should be. An often discussed best practice is to design events to be small and carry only the information about the change that triggered it. Following this approach usually produces town crier events (also known as event notifications^{[7](#)}). The term comes from real town criers, who typically inform people of important announcements by shouting them in the streets. In medieval England, most people couldn't read newspapers; town criers were an essential way of communicating news. The town crier shouted the central part of the announcement, and an additional note was usually posted at the local inn in case anyone would like to have more information. Town crier events follow a similar approach; events are published with minimal details, and the subscribing services

can request additional information about the change to the producing service. This section will discuss this approach to design event schema and illustrate it with a use case using an order submission process.

Let's work through a similar example to the ones we discussed in the previous chapters. The order service manages the orders of an eCommerce platform. Users submit orders and can edit their information as long as the order doesn't reach a given state. In this case, the user changed the street door number of the order's address. Listing 8-2 illustrates a possible event reflecting this change using only minimal information.

```
1    OrderStreetNumberChanged
2    {
3        OrderId: 3621,
4        AddressStreetNumber: 21
5    }
```

Listing 8-2 Example of an OrderStreetNumberChanged event

Keeping the event schema small and single-purpose is excellent for the overall system. Small events are easy to process and are faster in terms of serialization and deserialization. They also usually have less impact on the message broker and the service's resources. An often good approach aligned with the DDD mindset is to design events according to the user's intent. In this case, think of it in designing an OrderUpdated vs. OrderStreetNumberChanged; the first we don't know which was the exact intent of the user, and the latter represents cleanly the intent of changing only the address street number. Designing events this way captures the business value of the process and imbues events with domain value. The event stream will cleanly capture the user's intent and the flow of the business process. The domain itself doesn't sit solely on the service's logic but in the event stream as well. Retaining this domain value can be highly beneficial as an audit or when rebuilding different views of the data according to business needs. However, depending on the use case, this minimal design might also pose some challenges to the consumers.

The event has only the essential information related to the change, the id of the order that the user changed and the new street number. Let's say the inventory service handles order events to reflect stock changes and calculates the best warehouse to satisfy the order. Every time the order address changes, it has to recalculate the best warehouse and reflect the changes in the product's stock. The routing algorithm to calculate the best warehouse to satisfy the order is based on its full address. In this case, this event schema design poses a challenge to the inventory service. The service needs

the complete address to calculate the best warehouse, but the event only has the new address street number. When handling a partial event, the inventory service would have to request the whole order's address from the order service, as illustrated in Figure 8-4.

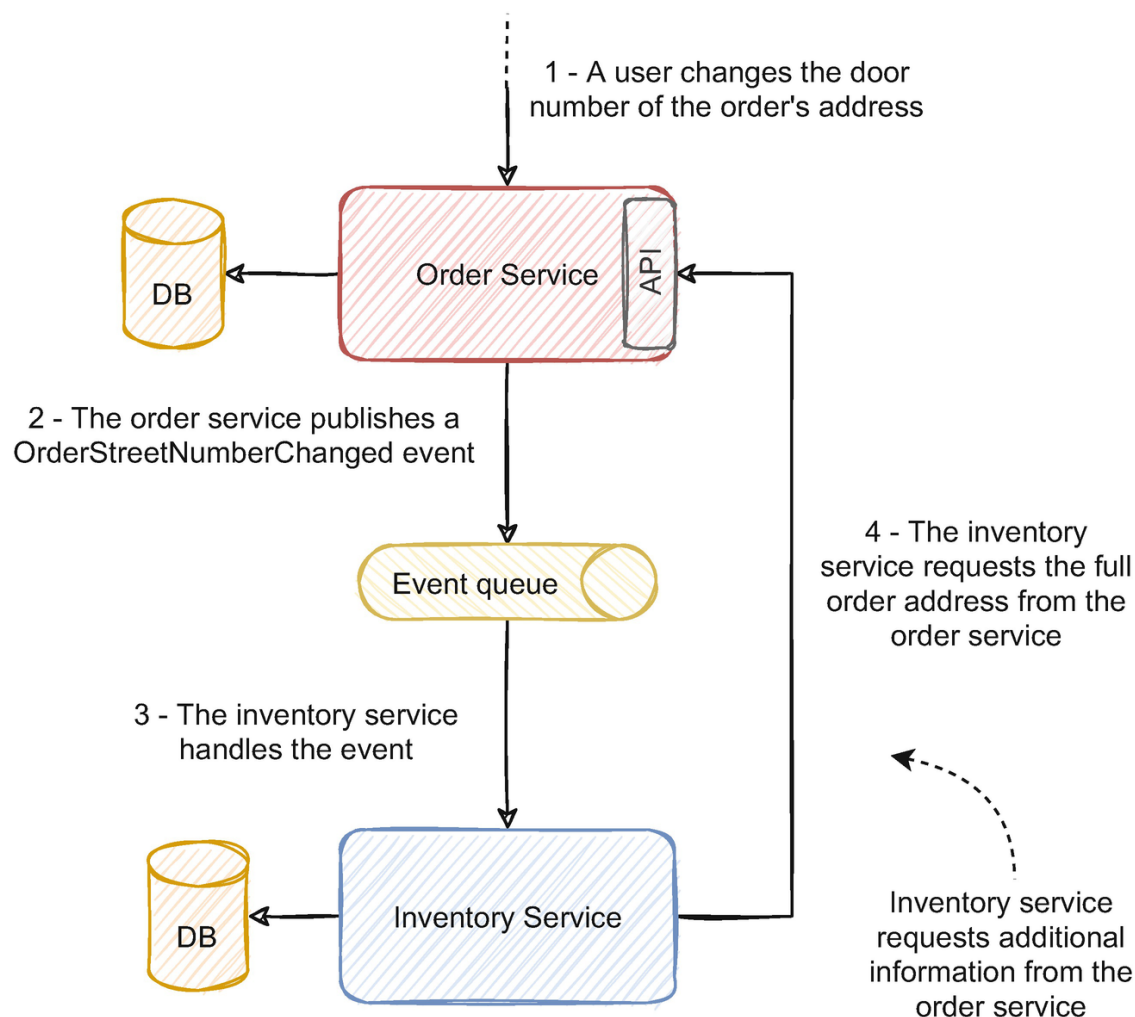


Figure 8-4 The interaction between the order and inventory service with a town crier event

The additional remote request to fetch the remaining address information has the impacts we discussed in the previous chapters. The inventory service now has a synchronous dependency on the order service, being susceptible to cascading failures. If the order service has an issue, the inventory service might be directly affected instead of just processing the events asynchronously. Scaling might also become an issue; if we need to horizontally scale the inventory service by adding more instances, it will trigger additional requests to the order service, which might need to be scaled as well. This topology is closer to the distributed monolith we discussed before. Consuming the event might also need additional complexity if the order service's read model is eventually consistent. For example, suppose the order service applies a CQRS pattern and asynchronously updates a read model API that the inventory service uses. In that case, the

information in the API might not be up to date with the event. Ideally, we want the consumers to be able to process events without recurring to external dependencies.

Partial and small events are usually helpful when consumers need to react to the specific change that triggers the event. For example, the order service could publish an `OrderUpdated` event every time the user changed anything in an order. Still, in this case, the only thing the inventory service cares about is the address. If the inventory service were consuming an `OrderUpdated`, it would have to understand what changed; any data in the order could have changed, not only the address. It would likely need to store some metadata of the order's address to understand the address changed and not something else in the order data to react accordingly. In this situation, an arguably better option would be to publish an `OrderAddressChanged` instead of an `OrderUpdated` or `OrderStreetNumberChanged`.

Town crier events are relevant when consumers need to react to the domain process that triggered the change. Designing them according to that change is a good approach, but we need to be aware to make the event sufficiently relevant for the consumers to rely on it without depending on synchronous dependencies.

8.4 Bee Events Pattern

An alternative to requesting additional data upon handling an event with minimal information is to keep an internal state about the event's entity. This approach produces an interesting effect where the event's state is persisted across several services. It is in many ways similar to the effect bees have on flower pollination. Pollination is the transfer of pollen grains between flowers by animal activity that generates fertilization. Pollination isn't carried deliberately by bees, it is the unplanned consequence of the bee's travels. Pollen sticks to the bee's body, and it is spread from one blossom to another as the bee moves. Bee events (also known as event-carried state transfer⁸) are fairly similar; consumers handle events and persist the event's data locally, actively spreading the event's state across several services, even though that might not be the producing sys-

tem's intent. This section will work through an example with bee events and detail the impacts on overall architecture and consumers.

In Figure [8-4](#), we discussed a practical use case using the order service. Upon receiving the `OrderStreetNumberChanged` event, the inventory service made a synchronous request to the order service to fetch the additional address information. In order to avoid the synchronous request's limitations we discussed before, the inventory service could save the address of each order internally. With each partial event published by the order service, the inventory service would update each order's address internally. The `OrderCreated` event would update the address for that order, the `OrderStreetNameChanged` would update the street name, the `OrderStreetNumberChanged` the street number, etc. This way, the inventory service could fetch the complete address from its local database to calculate the best warehouse to satisfy the order.

This solution preserves the decoupling properties of event-driven services. The services depend only on their resources to do their business logic without relying on external dependencies, effectively avoiding the drawbacks of synchronous requests. It also has a performance boost since fetching information from a local database is typically faster than a remote request. Storing a view of the order information also provides the opportunity to materialize it in the most beneficial way for the service. Each service would only store the relevant information for its use case in the granularity that it needed. One obvious consequence is the order information will be scattered across several services, as illustrated in Figure [8-5](#).

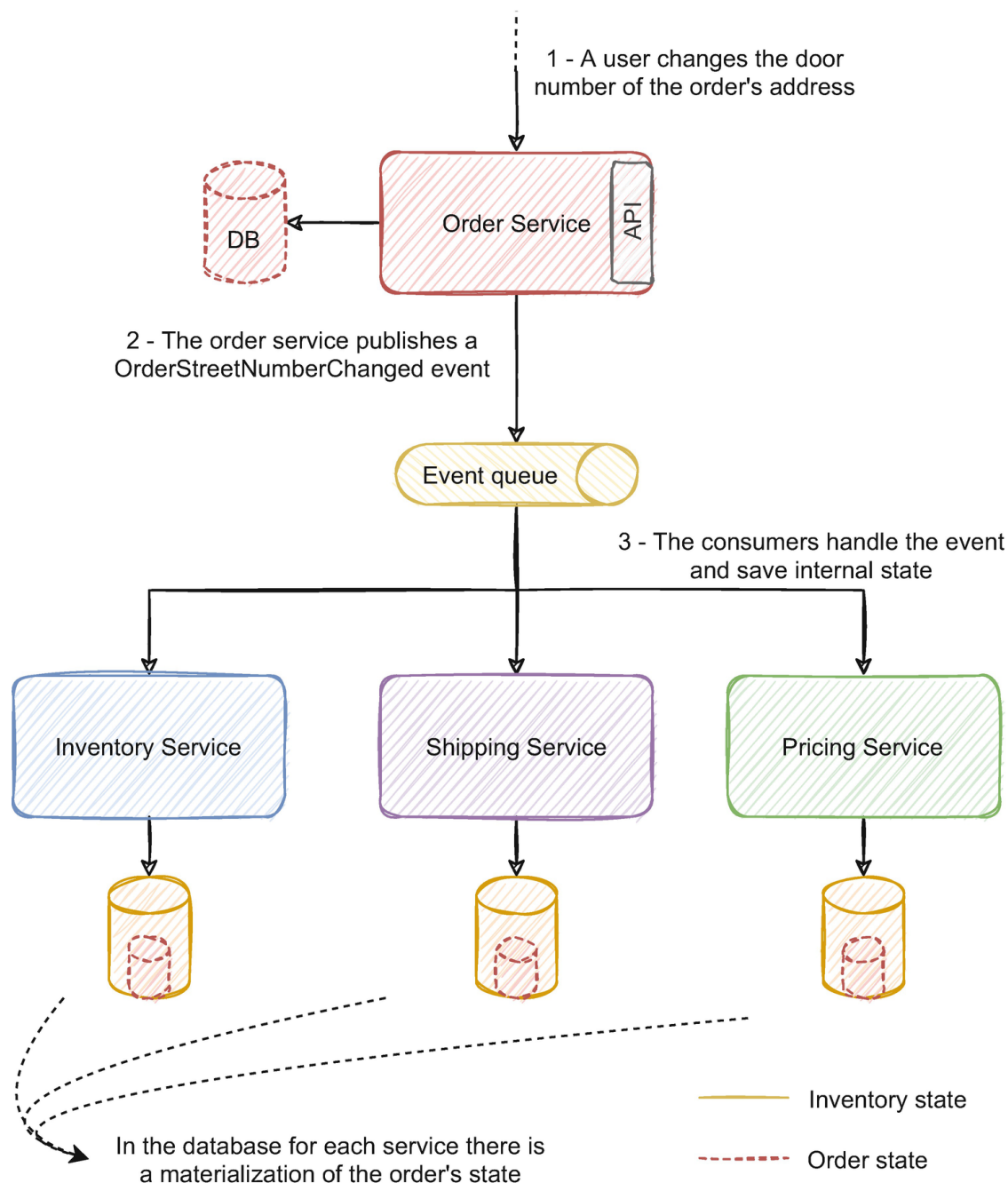


Figure 8-5 Several services persisting a view of the order information

In this case, the inventory, shipping, and pricing service need the order address to calculate the stock changes, ship the order, and calculate the country's taxes, respectively. To access the order's full address, they save it internally and keep it updated by applying each new event to their internal state. Bee events end up transferring the order state to each consumer.

Having the order state distributed across all consuming services has an impact on the service's resources; it will require additional database space, although currently disk space isn't as concerning as it used to be. The real challenge can be initializing and maintaining the data accurate-

ly. In traditional applications, this could be troublesome; however, as we discussed in Chapter [7](#), a new service could initialize the order state by reading the event stream from the start; new events would both trigger the business logic and update the order metadata. A bug in the order service might affect the other services' metadata, but guaranteeing the event stream is the source of truth; fixes are also published to the stream, which will automatically heal the consumer's state.

There are more complicated situations where the order service has to do a substantial restructuring of its internal schema, which can end up affecting other services. These situations can be troublesome to manage and end up needing the coordination of several teams. To avoid complex challenges, we need to guarantee each domain is segregated and has clear boundaries. The order state in the inventory service should be relevant only to the inventory service and not a copy of the order service. Anti-corruption layers should guarantee that only the relevant information exists in each boundary. Clear segregation should avoid internal schema changes in the order service to propagate to other services. If consumers rely solely on the event contracts, the order service can incrementally migrate its schema along with the event contracts (we will further discuss schema evolution in Section [8.6](#)).

The impact of having a small event schema can be solved with this approach. It is more aligned with the event-driven mindset than requesting the information with a remote request. Although usually requesting data through a remote request is easier and faster development-wise, this solution tends to be more sustainable when applied frequently. It preserves the services' domain segregation and fosters more resilient properties.

8.5 The Event Schema Goldilocks Principle

In Sections [8.4](#) and [8.5](#), we discussed how a small event schema can be beneficial to each service but can impact the overall architecture. We could easily argue if it was worth designing a small event schema if it affects the architecture in a way that makes the consuming services request or store a view of the data. This section will discuss a different approach by finding a middle ground between large and small event schema.

The Goldilocks principle⁹ also applies to event schema; we should avoid large events and small events can be challenging to consume, but in some use cases, the event schema size can be just right. Let's continue with the example we discussed in Sections [8.3](#) and [8.4](#); using town crier or bee events solves the challenge of consuming a partial event. In event-driven architectures, teams typically have ownership of different services usually related to their domain (as discussed in Chapter [4](#)). Different teams might maintain the order service and the inventory service, and it is easy for the team managing the order service to design events most beneficial to their context, in this case for publishing. Publishing small events is often easier and more straightforward from the producer perspective; as we discussed, the issue is in the consumer. An often ignored and essential consideration while designing event schema is always having consumers' needs and use cases in mind. Involving current and future consumers in the discussion can avoid intricate solutions. For example, instead of the order service publishing an `OrderStreetNumberChanged` event, it could publish an `OrderAddressChangedEvent` as illustrated in Listing [8-3](#).

```
1  OrderAddressChanged
2  {
3      OrderId: 3621,
4      StreetName: "Palm Avenue",
5      StreetNumber: 21,
6      State: "FL",
7      City: "Tampa",
8      Postal: 33619,
9  }
```

Listing 8-3 Example of an `OrderAddressChanged` event

Instead of publishing only the information that changed, for example, the street number, the order service could publish the order's full address, even though the only information the user changed was the street number. Designing a more comprehensive event adapted to the consumer's needs can be advisable in most use cases. In this example, the inventory service would avoid any external dependency or additional internal state, as pictured in Figure [8-6](#).

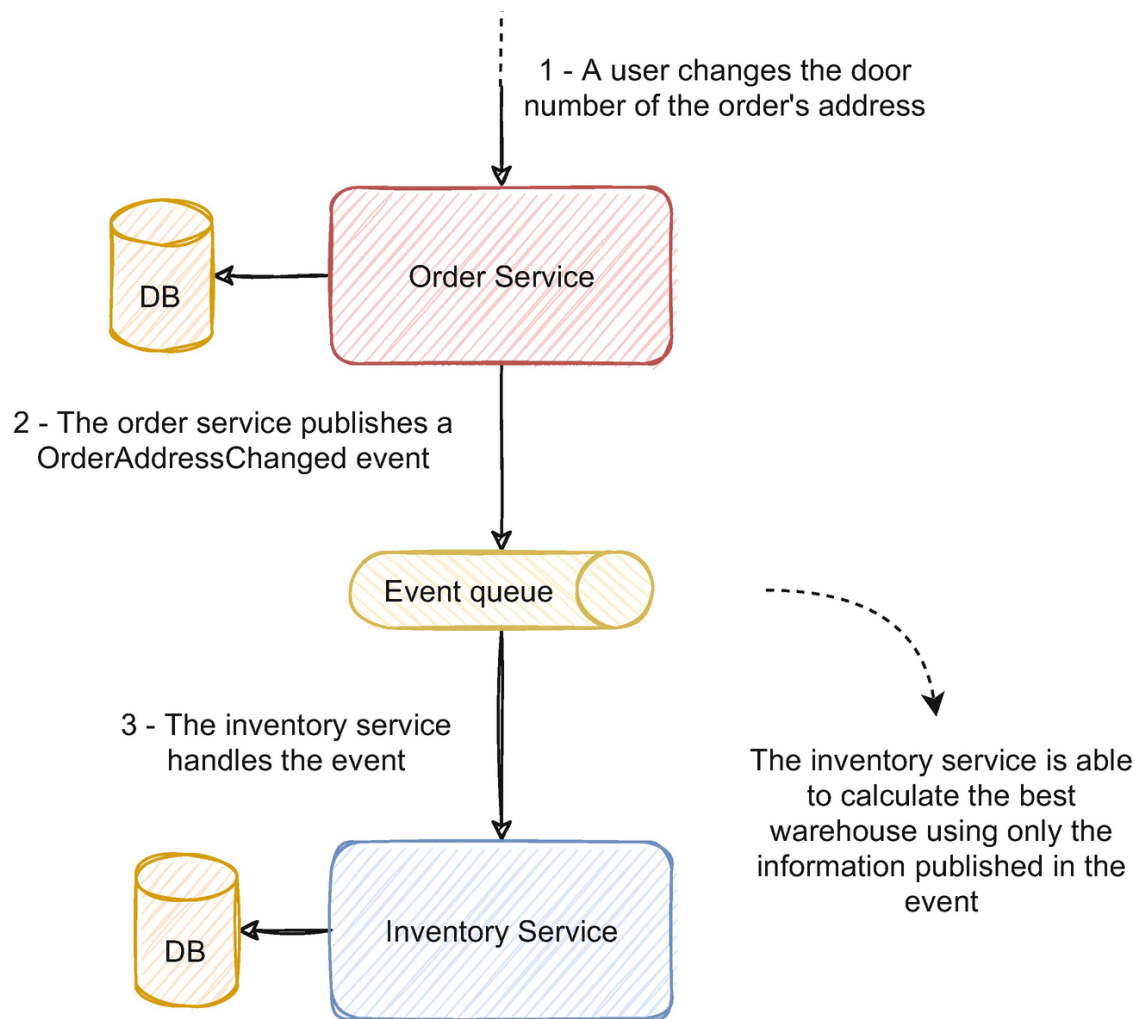


Figure 8-6 The consumer is able to apply its domain logic using only the event's information

This way, the inventory service would be able to calculate the best warehouse using only the event's information, avoiding the complexity overhead of the other solutions. Depending on the use case, it might be harder to enrich the event with additional information on the producing side, and it can be highly beneficial on the consumer side.

The caveat is to not add too much information to the event, especially information not in the service's scope or domain. If the inventory service needs the country's geographical position, which is another service domain, adding that information in the order event would be inadvisable. Large events might also reflect that the service has too much responsibility and might need to review its boundaries.

The event might also lose its domain meaning; an `OrderAddressChange` carries less domain value than an `OrderStreetNumberChanged` since it's no longer clear which field the user changed. If consumers need

to react specifically to changes in the street number, the latter is typically more advisable than the first.

As a consumer, you might not be able to request changes to the existing events. For example, if they come from an external tool or a legacy application or the additional information requires breaking changes, it might not be practical to adapt them to the consumer needs. In those situations, you might have to follow one of the town crier or bee events patterns. An arguably sound reasoning would be to give precedence to bee events and avoid direct dependencies between services. As a faster, more tactical approach, town crier events might be more advisable since they are typically quicker to implement but avoid applying it frequently.

Finding a middle ground or the Goldilocks sweet zone might be tricky and require a more profound knowledge of the overall architecture, thus the importance of involving the consumers. We often argue if the solution will survive the test of time and try to do futurology of possible use cases. Still, sometimes a more pragmatic approach of the current use cases might wield better results than optimizing the solution for use cases that might never happen. Try to follow an incremental approach, involve the existing consumers and design for those use cases, and then iterate on the most appropriate design.

8.6 Denormalized Event Schema

As the microservice architecture becomes more complex and services become more fine-grained, you might encounter the challenge of needing the information of several event streams. Handling several event streams can be a challenge due to the difficulty of managing concurrency and consistency of the different data sources. This section will discuss how we can overcome this challenge by denormalizing the event schema.

Data being distributed across different components is a consequence of moving to a microservice architecture. As we discussed, this has several advantages but also proves to be one of the most difficult challenges of distributed architectures. Although many use cases adapt well to this distributed nature, some use cases don't. Instead, they need a more compre-

hensive and aggregated view of the information. Providing an aggregated and denormalized view can prove extremely difficult without complicated solutions or falling into the trap of building a monolithic service. Also, the strategies we discuss in Chapter [6](#) are extremely useful to solve concurrency and ordering; however, they require the consumption of a single event stream. If a service consumes data from different services, the various events can have different granularities and routing keys.

For example, the shipping service is in charge of shipping orders from a warehouse to the user's address after the payment is processed. The service would likely need to consume events from three different services: the order, inventory, and product service. The service would require a denormalized view of order with the information of the products, stock, and warehouses to process each order. This example is illustrated in Figure [8-7](#).

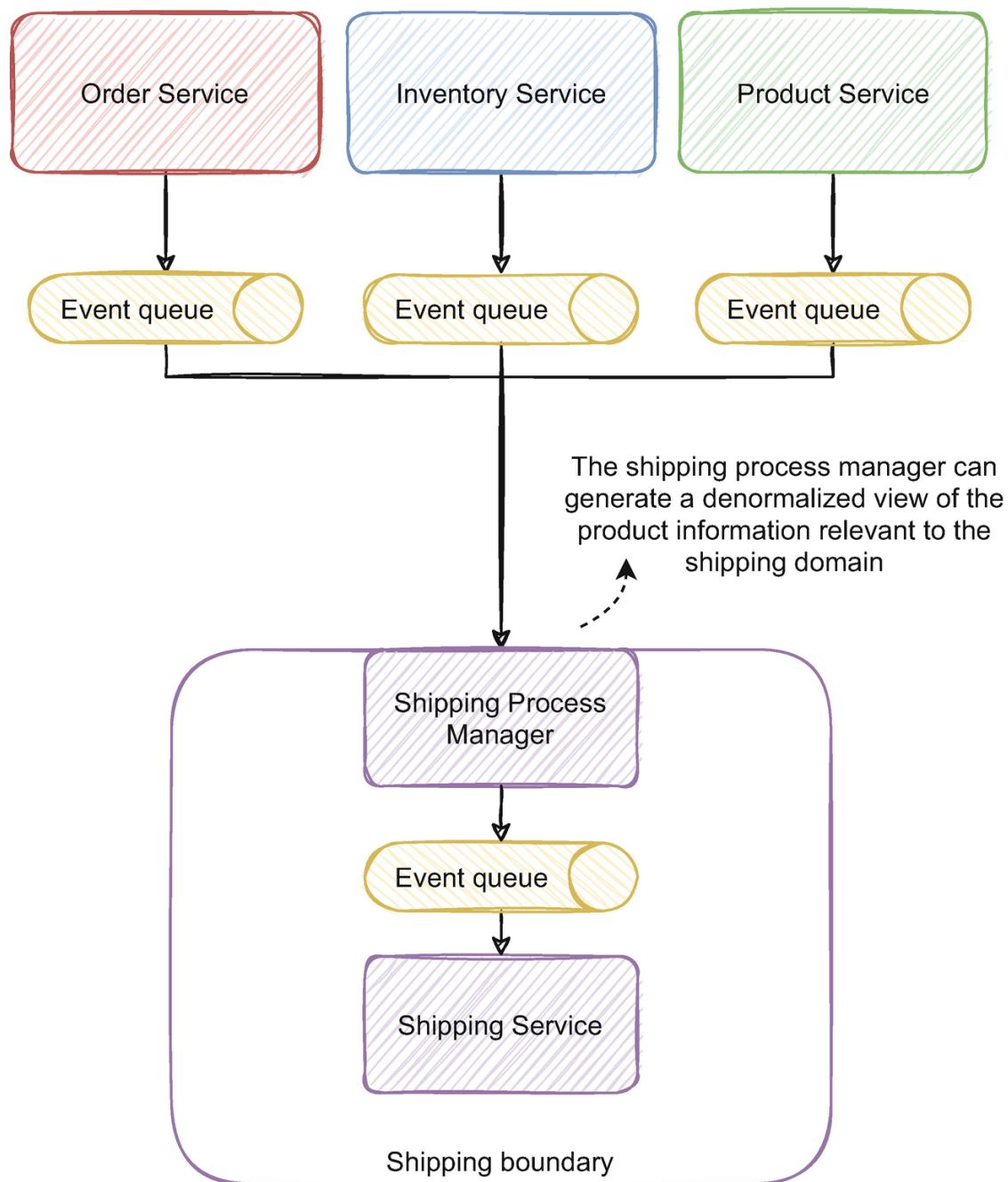


Figure 8-7 A process manager can make a denormalized view of an entity relevant for a boundary

To build an aggregated view of the order, we can use a process manager to join the information from the different streams in a larger denormalized schema. The order service events would only contain information about the order, the inventory events about the stock, and the product events about the products. The process manager would be responsible for merging each order's information into a larger event relevant to the shipping domain logic.

We mentioned before how each boundary could have a view of the entities of other boundaries with only part of the information relevant to that boundary. However, the inverse is also true; the view of an entity can

also be larger. The order entity in the shipping boundary is larger and more comprehensive than the remaining services, reflecting in larger denormalized event schemas.

The process manager acts as an anti-corruption layer and enriches the order data. This way, the shipping service that owns the shipping domain logic can process the orders without dealing with the complexity of merging several event streams. The internal order event stream in the shipping boundary also effectively reflects the view of the order inside the shipping boundary and might be useful for other services, such as a read model.

The caveat is guaranteeing the process manager doesn't accrue extensive responsibility. The process manager is in charge of adapting and merging information and creating a denormalized view of the events; it shouldn't have substantial domain logic if any. Using denormalized event schema often produces larger events which have the challenges we discussed in Section [8.3](#). Still, the advantage of applying event routing and dealing with concurrency and order by design often surpasses the impacts of larger events.

8.7 Schema Evolution in Event-Driven Microservices

Event schema is as essential as API contracts and often as difficult to change. Inserting a breaking change on an API can be highly complex and require the alignment of all consumers. The same applies to event schema, although there are some strategies we can follow to ease that transition. This section will discuss which kinds of changes are safe to do on event schema and detail those strategies working through some practical use cases.

Event schema changes through time; we might need to add new information or descope old one in light of new use cases. The changes to the event schema can have larger or lesser impacts depending on the change's nature. More often than not, we can't simply change schema and consumers at the exact same time. Changes to the event schema are pro-

longed during a period where services adapt to the new changes. It is also vital that anyone changing event schema is aware of the impacts. Enforcing compatibility rules is also a good approach to avoid accidental or unforeseen impacts. Some serialization systems, like Avro,¹⁰ can implement these rules and make them transparent across producers and consumers and avoid harsher changes in the schema.¹¹ There are typically four compatibility types when evolving schema: backward, forward, full, and no compatibility.

8.7.1 Backward Compatibility

Consumers can read events produced with an old schema with the new schema. It allows consumers to read events with both old and new schema. Let's illustrate an example of a backward compatibility change with the event in Listing [8-4](#).

```
1   OrderCreatedEvent
2   {
3       OrderId: 15251212,
4       ProductId: 1147421,
5       Address: "9980 Rock Maple Street",
6       Quantity: 1,
7       OrderedAt: "2021-01-23T18:25:43.511Z",
8       UserId: 12168,
9       UserName: "John Allan"
10  }
```

Listing 8-4 Order created event example

Let's say we create a new service to manage the user information and no longer want to publish the user's name in the order events. Removing the `UserName` field would be a backward compatible change. Consumers that have already updated to the new schema (the `OrderCreatedEvent` without the `UserName`) would be able to consume events with an old schema (event with a `UserName` field); they would ignore the old field.

Backward compatible changes can be especially useful when producers and consumers design a new schema and make it available before the functionality. The event schema is available; consumers can update to the new schema even though the producers are still publishing events with the old one. Once the functionality is available, the service can publish the new schema since the consumers were already updated.

It also might be helpful when replaying events from a stream with an old event schema. If the event schema evolved to different versions and wasn't upscaled (old events converted to the latest schema), backward compatible changes allow consumers to reprocess old event streams if needed.

8.7.2 Forward Compatibility

Consumers can read events produced with the new schema with an old schema. Even though consumers didn't update to the new schema, they can still receive events with the new schema.

For example, in Listing [8-4](#), if we needed to add a country field in the event, it would be a forward compatible change. In most situations, consumers with the old schema (without the country field) would be able to consume events with the new field; they can simply ignore it.

Forward compatibility is especially relevant when producers develop new features and publish events with the new schema, and consumers will be updated in the future. Since this is frequently how new features are implemented, guaranteeing forward compatibility is often incredibly useful to manage new developments.

8.7.3 Full Compatibility

Full compatibility is the combination of backward and forward compatibility. Consumers can read new events with old schema and old events with new schema. Since full compatibility requires the ability to define a default value for removing or adding new fields, it is only supported by some message formats. The JSON format, for example, doesn't support fully compatible changes, unlike Protobuf and Avro, for instance, which support optional fields.

Imagine if we wanted to add a description field in the event in Listing [8-4](#). It is a forward compatible change, but it isn't backward compatible since consumers with a new schema wouldn't know what to fill in the description field when receiving old events without that field. However, if

we were able to define a default description when the description isn't present, that would make the change fully compatible. The same can be said for non-forward changes when we remove a field.

8.7.4 No Compatibility

There might be changes that aren't in any way compatible. Some updates to the schema change it in a way they are utterly incompatible for consumers with a different version. For example, if we changed the `UserId` in Listing [8-4](#) to an UUID, it would be an incompatible change. Consumers with the new schema wouldn't be able to handle old events, and consumers with the old schema wouldn't be able to handle new events.

Incompatible changes can be hard to manage since, with a typical topology, consumers and producers would have to update the schema simultaneously. In a distributed architecture, this is often impossible. Since each service is independent and has segregated deployment procedures, coordinating a simultaneous change is exceptionally challenging.

8.7.5 Managing Changes

Forward, backward, and full compatible types are illustrated in Figure [8-8](#). These compatibility types with careful coordination support the incremental evolution of the schema. If we support any of these three compatibility types, we can manage schema evolution without substantial changes, just coordination.

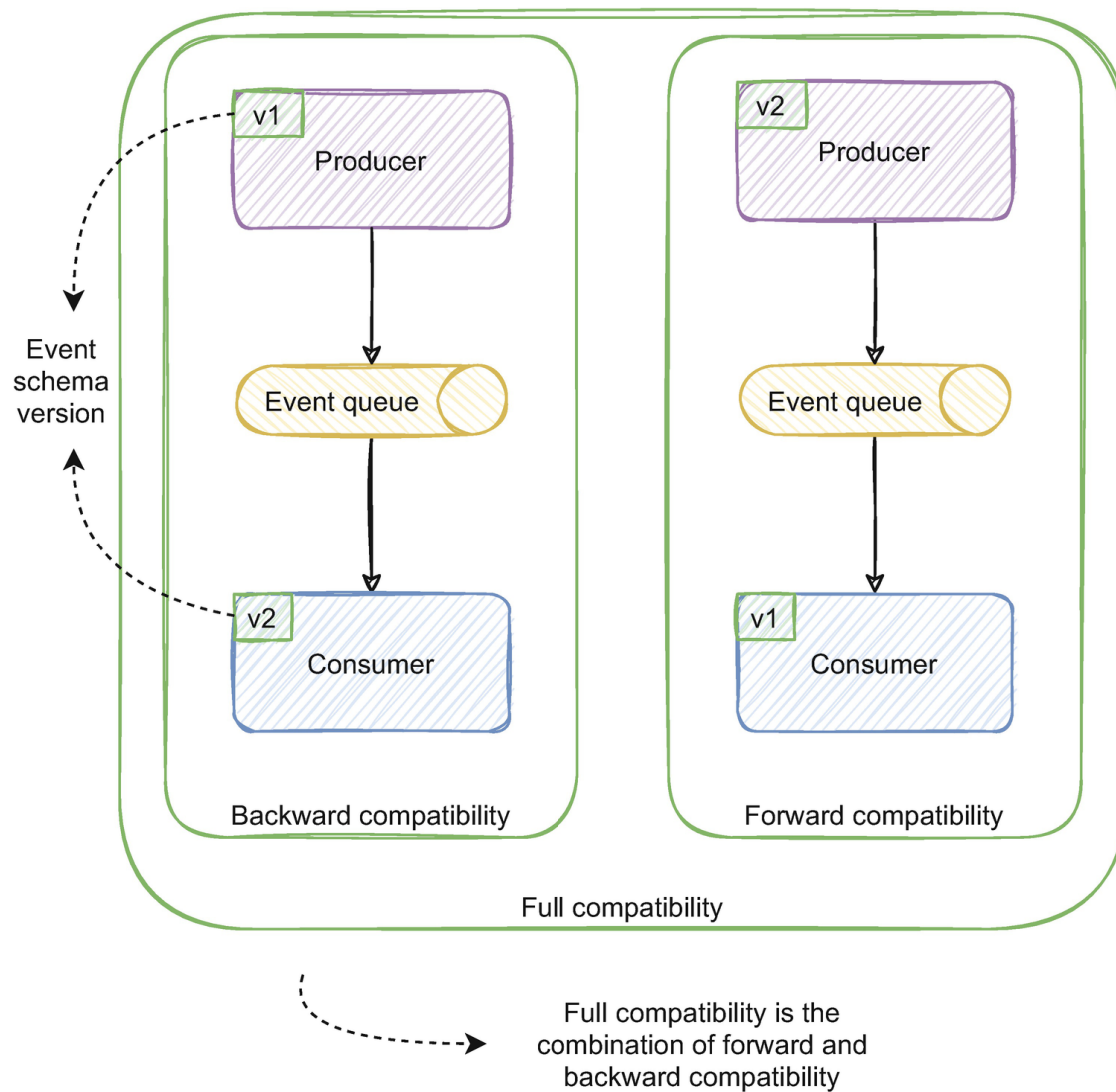


Figure 8-8 Backward, forward, and full compatibility types

But what about cases where we need more structural and non-compatible changes? For example, in Listing 8-4, how could we change the address field from a string to a structured object containing the address data in segregated fields? Listing 8-5 exemplifies the new event schema.

```

1  OrderCreatedEvent
2  {
3      OrderId: 15251212,
4      ProductId: 1147421,
5      Address:
6      {
7          "StreetName": "Rock Maple Street",
8          "StreetNumber": 12,
9          "City": "Orlando",
10         "State": "FL",
11         "Country": "USA",
12     }
13     Quantity: 1,
14     OrderedAt: "2021-01-23T18:25:43.511Z",

```

```
15         UserId: 12168,  
16         UserName: "John Allan"  
17     }
```

Listing 8-5 Order created event example

The change in the address field type is an example of an incompatible change; the only way to apply it to the event is to change consumers and producers simultaneously. However, changing both consumers and producers is likely risky and impossible or impractical. Instead of applying the change in a single transition, we can divide it into two steps: transforming the incompatible change into a forward compatible change and then into a backward compatible change.

We can add the new address structure along with the existing one. Adding a new field is a forward compatible change; consumers with the old schema can still process this event. Publishing events with the new field allows consumers to adapt to the new structure sequentially. We might have several different applications consuming our events; each of them will have different priorities and roadmaps; by exposing a new field, we give the flexibility to each consumer to change to the new structure in the correct timing. Once all consumers adjust to the new format, we can remove the old address field being a backward compatible change.

A reasonable approach is to define a max period to maintain the old field and let consumers adapt. Having a period instead of doing the change in a single moment is easier to manage; we let consumers adjust to the new data incrementally. If anything goes wrong, they can roll back to the old field, being also a safer and more resilient approach.

8.7.6 Event Stream Versioning

Applying the incremental evolution we discussed before can be a good approach. Still, it might not be practical when we need to do a substantial redesign of the event schema or discontinue an event and replace it with a new one. In those situations, we need a more sustainable approach to manage schema evolution. This subsection will detail how event versioning can be a relevant approach to those situations.

Migrating schema can be risky, even traumatic when not managed correctly. As with most developments we deploy to live, we want to avoid big bang releases. We want to guarantee changes are incremental, and when they go wrong, we can roll back to the previous version. One way to safeguard these guarantees is to use stream versioning. Each event schema version is published to one stream; when we need to evolve the schema and create a new version, we create a new stream with the new schema. Figure 8-9 illustrates an example of this approach.

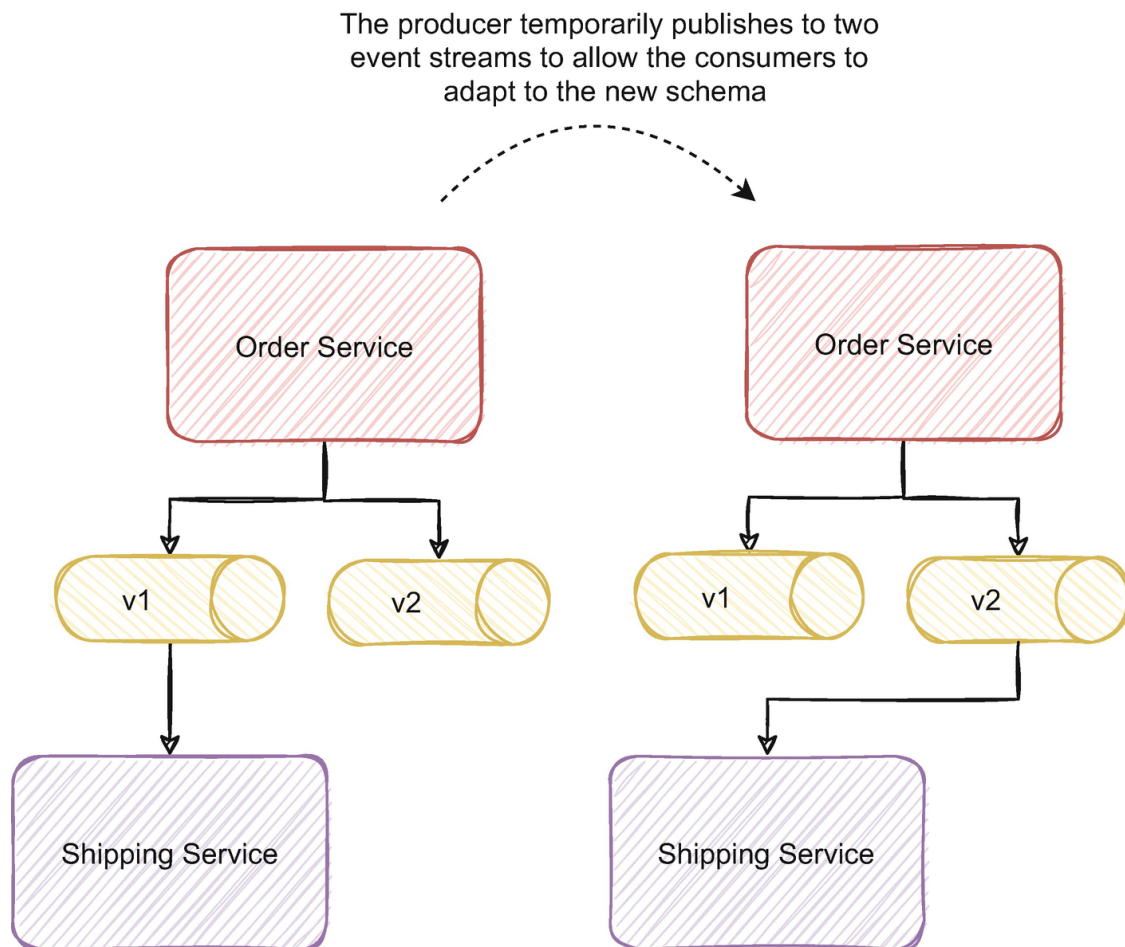


Figure 8-9 Using one stream per event schema version

The order service evolves the schema of an event from version 1 to 2. Initially, it starts to publish the version 2 events to a separate stream while also maintaining the version 1 stream. The consumers, in this case the shipping service, can make the required changes to adapt to the new schema. Once all consumers use the new event schema, the order service can descope the old schema and only use the latest one. The order service should also republish all the existing events in the old schema stream to the new one in order to guarantee the event history and that the new stream has all relevant data.

This approach allows the consumers to gradually move to the new stream without a big bang release. If anything goes wrong, they can also roll back the consumption to the old stream. Publishing to two streams has the disadvantage of guaranteeing every event is published to both streams or none; we have to ensure both are consistent and have the same data. To do so, we can use one of the strategies we discussed in Chapter [7](#). Kafka also supports transactions when publishing events to two streams^{[12](#)} simultaneously.

8.7.7 Using a Downscaler/Upscaler When Evolving Event Schema

Profound changes in the event schema are often associated with a change in a domain concept or leaving some kind of legacy representation we no longer want to support. Using stream versioning has the disadvantage of the service in charge of the domain having to maintain the logic to generate the old event schema until all consumers adapt to the new one, which can take a considerable amount of time. An alternative to this approach is to segregate the legacy logic into a separate component commonly known as a downscaler. This subsection will work through the same use case in Figure [8-9](#) but with a downscaler approach.

Sometimes it can be challenging to maintain the old schema when we do substantial changes. For example, imagine if we moved the logic that managed the user's address to a service that manages users. We would need to remove the address information from the event in Listing [8-5](#). Publishing events with the old schema would require maintaining (or fetching) the address information from that service. It wouldn't be beneficial to the order service to maintain that logic until all consumers adapt to the new paradigm. In order to maintain the responsibility of the order service focused on managing the orders, we could delegate the legacy logic to a different component, a downscaler. Figure [8-10](#) illustrates the same example but with a downscaler.

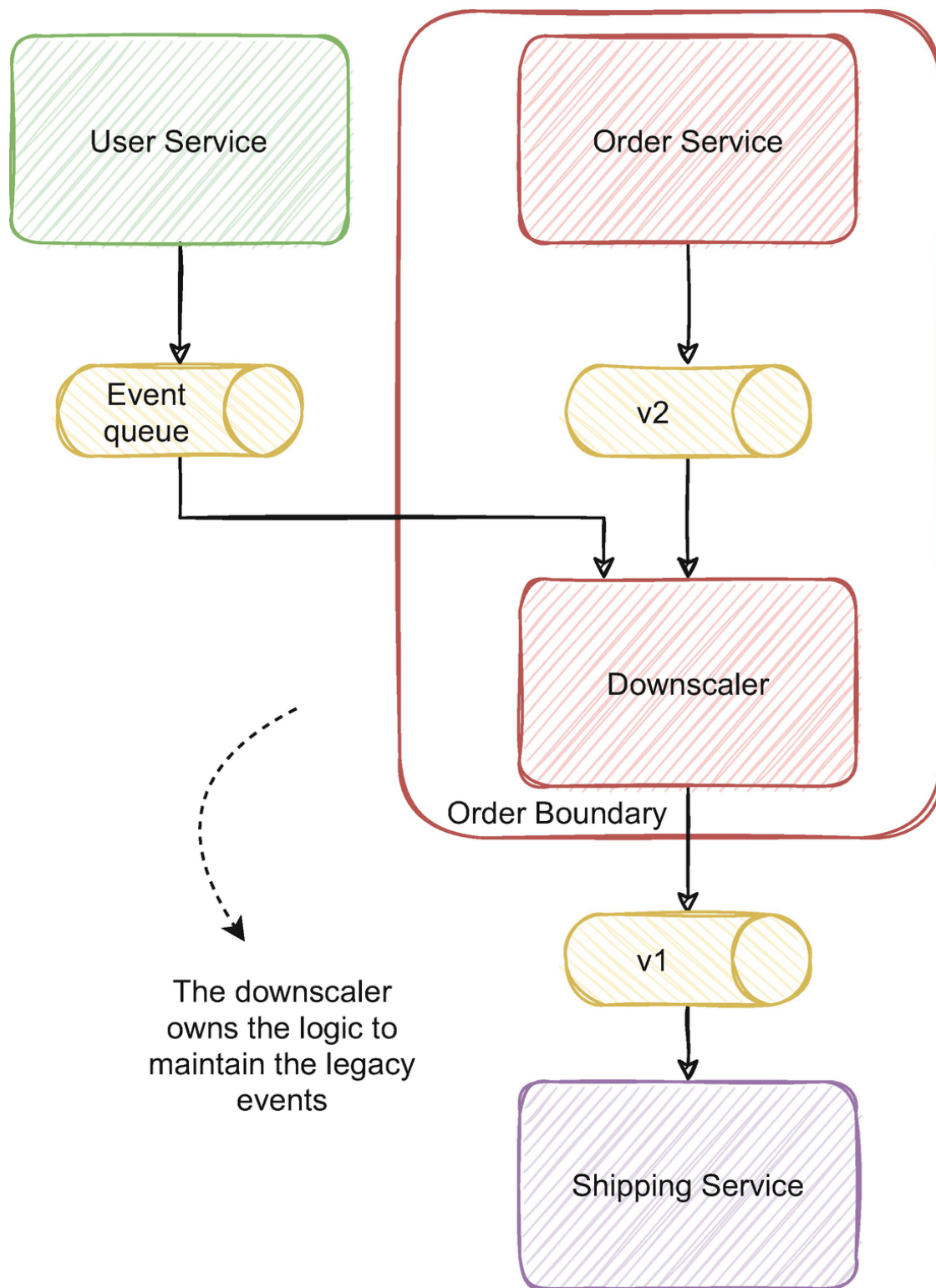


Figure 8-10 Using a downscaler to segregate the logic to maintain the events with the old schema

The downscaler uses the information from the new order events and the user events to maintain the old order events. This pattern has the advantage of keeping the order service (the owner of the domain) clean and focused only on the vision we have for the domain, without the additional complexity of dealing with legacy events. Once all the consumers use the new version, we can discontinue the downscaler. The downscaler only has the responsibility to maintain the old schema and is only relevant

while they are needed. This approach is an example of leveraging the high decoupling nature of event-driven architectures and the flexibility of microservices to segregate legacy logic to contained services. Once the old logic is no longer needed, we only discontinue the service; we don't have to change anything in the order service. It can evolve freely without waiting for the consumers to adapt. The inverse (adapting an old version schema to a newer version) is also possible with the same approach using an upscaler.

This pattern is relevant when consumers are expected to take a considerable amount of time to adapt to the new schema. Otherwise, it might not be worth the effort of creating and releasing an individual service. We also don't need to create a separate component; the downscaler can be a separate module on the order service as long as we can maintain it decoupled from the surrounding logic. The pivotal point is when we want to discontinue the logic, it should be straightforward and not affect the main service logic.

8.8 Summary

- Event storming can be a valuable strategy to model your business process according to the flow of events. It provides a collaborative way to obtain a higher level of understanding of the domains and temporal business flow. The event storming session's output events are also valuable to design the service's domain events and technical schema.
- There is common information relevant to most events, like the correlation id or version. These generic properties are often best shared with a generic structure like an envelope or event headers.
- Consumers handling town crier events are notified of changes in an upstream domain. The service requests additional information when the event schema doesn't have all the required data. Partial and small events are usually useful when consumers need to react to the specific change that triggers the event. But when the consumer needs to request additional information through synchronous requests, other options are typically more beneficial.
- Consumers handling bee events keep an internal state about the event's entity. It is more aligned with the event-driven mindset than

requesting the information through a remote request but has the disadvantage of spreading the event's state across several services.

- Having the consumer's needs and use cases in mind is a pivotal concern to design event schema. It can significantly simplify the architecture and avoid complicated solutions on the consumer's side.
- Consumers should be able to make the decisions they need using only the information in their domain and the event's information. This approach will contain the dependencies between each service and enable a truly decoupled scalable architecture.
- We can use a process manager to denormalize event schema and enrich events with additional information to simplify consumers.
- There are four compatibility types: forward, backward, full, and no compatibility. Forward, backward, and full are useful to evolve the schema incrementally without substantial changes in the ecosystem. The no compatibility type requires more complex solutions.
- To manage more profound and structural changes in event schema, we can use stream versioning or a downscaler/upscaler.

Footnotes

¹ Book by Alberto Brandolini, "Event Storming," www.eventstorming.com/book/

² Talk in NewCrafts Conferences, Alberto Brandolini, "Introducing EventStorming," June 9, 2015, <https://vimeo.com/130202708>

³ Talk in Saturn 2017, Paul Rayner, "EventStorming: Collaborative Learning for Complex Domains," May 31, 2017, www.youtube.com/watch?v=vf6x0i2d9VE

⁴ Book by Dave Gray, Sunni Brown, and James Macanufo, "Gamestorming: A Playbook for Innovators, Rulebreakers, and Changemakers," August 17, 2010, www.amazon.com/Gamestorming-Playbook-Innovators-Rulebreakers-Changemakers/dp/0596804172

⁵ Full article by Alexei Zenin, "How to Choose Between Strict and Dynamic Schemas," November 9, 2020, www.confluent.io/blog/spring-kafka-protobuf-part-1-event-data-modeling/

⁶ Project in GitHub, <https://github.com/cloudevents/spec/blob/master/spec.md>

[7](#) Article by Martin Fowler, “What do you mean by “Event-Driven”?”, February 7, 2017, <https://martinfowler.com/articles/201701-event-driven.html>

[8](#) Article by Martin Fowler, “What do you mean by “Event-Driven”?”, February 7, 2017, <https://martinfowler.com/articles/201701-event-driven.html>

[9](#) Wikipedia page at https://en.wikipedia.org/wiki/Goldilocks_principle

[10](#) Avro home page in <http://avro.apache.org/>

[11](#) Article in confluent documentation, “Schema Evolution and Compatibility,” <https://docs.confluent.io/platform/current/schema-registry/avro.html>

[12](#) Article by Apurva Mehta and Jason Gustafson, “Transactions in Apache Kafka,” November 17, 2017, www.confluent.io/blog/transactions-apache-kafka/

[Support](#) [Sign Out](#)